

FORJA

Documentación técnica detallada del proyecto

Generador y seguimiento de rutinas de gimnasio con Inteligencia Artificial

Proyecto Final · Automatización e IA para MVPs · MBA UAI · Track B

Integrantes: Javier Venegas · Raimundo Bravo · Tomás Núñez · Zhen Chi

Este documento explica, paso a paso y por secciones, cómo está construido FORJA: desde la página web hasta la automatización con IA, incluyendo el detalle de cada flujo de n8n (qué hace, por qué se usa y para qué sirve).

1. ¿Qué es FORJA?

FORJA es un producto (MVP) que resuelve un problema concreto: quien empieza en el gimnasio no sabe qué rutina seguir, se siente perdido y no puede pagar un entrenador personal. FORJA genera una rutina personalizada con IA, la envía por correo junto con una hoja para registrar los pesos, y luego ajusta automáticamente las cargas según el progreso real del usuario.

La idea en una frase:

Un formulario web guarda datos en Google Sheets → n8n detecta el dato nuevo → una IA (Gemini) genera o ajusta la rutina → el resultado llega por correo, con una hoja para registrar pesos que la IA luego usa para mejorar la rutina.

2. Arquitectura general

El proyecto conecta 5 piezas. Cada una cumple un rol claro:

- **Landing page (Front):** la web donde el usuario rellena los formularios. Publicada en GitHub Pages.
- **Google Apps Script (el puente):** recibe los envíos de los formularios y los escribe en Google Sheets.
- **Google Sheets (Base de datos):** almacena los datos de los usuarios y sirve de disparador para n8n.
- **n8n (la automatización):** orquesta todo: detecta datos nuevos, llama a la IA, crea hojas y envía correos.
- **Google Gemini (la IA):** genera y ajusta las rutinas.

¿Por qué se usa n8n?

n8n es una herramienta de automatización visual: permite conectar servicios (Google Sheets, Gmail, una IA) mediante 'nodos' sin escribir un backend completo. Se eligió porque hace visible el proceso, maneja los errores de forma sencilla y se integra con Google y con la API de Gemini. Es el 'cerebro' que hace que todo ocurra solo, sin intervención manual.

Flujo de datos (resumen):

Formulario → Google Sheets → n8n se dispara con la fila nueva → Gemini genera/ajusta la rutina → se crea o se lee la hoja de seguimiento del cliente → el resultado llega por correo. Un flujo aparte (Error Trigger) vigila y avisa si algo falla.

3. Componente 1 — La landing page (Front)

Es un único archivo **index.html** con diseño oscuro y acentos neón (verde lima / naranja), pensado para principiantes de gimnasio. Contiene **dos formularios**:

- **Formulario inicial:** nombre, correo, edad, peso, altura, nivel, días disponibles, tiempo por sesión, meta y lesiones. Con esto se genera la primera rutina.
- **Formulario de seguimiento:** correo, cómo le fue, peso actual, sensaciones, nueva lesión y cambio de meta. Con esto se pide un ajuste de la rutina.

Incluye además una sección de **planes** (modelo freemium: gratis con 1 seguimiento, o FORJA Pro a \$4.990/mes con seguimientos ilimitados) y una sección que explica el valor del seguimiento. El formulario de seguimiento envía un campo oculto (Formulario = seguimiento) para poder distinguirlo del inicial.

¿Por qué así?

La landing es el **touchpoint** del usuario: el punto por donde entra a la solución. Se hizo como página web para que sea accesible desde el móvil sin instalar nada, y con dos formularios porque el producto tiene dos momentos: crear la rutina y luego ajustarla.

4. Componente 2 — Google Sheets (Base de datos)

Todo el proyecto usa un único Google Sheet con varias pestañas. Es la base de datos del MVP: guarda los datos del negocio y además sirve de **disparador** para n8n.

Pestaña	Qué guarda
general	Cada rutina solicitada (datos del formulario inicial).
seguimiento	Cada solicitud de ajuste (datos del formulario de seguimiento).
hojas_clientes	La "libreta": mapa correo → ID de la hoja de seguimiento de cada cliente.
Hoja por cliente	Se crea automáticamente. Tiene los ejercicios + columnas vacías (Peso usado, Reps hechas, Dificultad) para que el cliente registre.

Nota: los nombres de columna del formulario son 'simples' (sin espacios ni paréntesis) para que las expresiones de n8n funcionen directamente.

5. Componente 3 — Google Apps Script (el puente)

Los formularios de la web no escriben directo en el Sheet: envían los datos a un pequeño programa (Google Apps Script) publicado como 'aplicación web'. Ese programa (función doPost) recibe el envío y lo escribe en la pestaña correcta:

- Si el campo Formulario es 'seguimiento' → escribe en la pestaña **seguimiento**.
- En cualquier otro caso → escribe en la pestaña **general**.
- Rellena solo la columna 'Marca temporal' con la fecha/hora del servidor.

¿Por qué así?

Es la forma más simple y gratuita de conectar un formulario HTML con Google Sheets sin un servidor propio. El enrutamiento por el campo 'Formulario' permite que los dos formularios compartan el mismo punto de entrada.

6. Componente 4 — Publicación en GitHub Pages

La landing se sube a un repositorio de GitHub y se activa GitHub Pages, que la publica en una URL pública y gratuita. Así cualquier persona (por ejemplo, un compañero probando la app) puede acceder desde su móvil. La entrega del proyecto vive en un repositorio privado aparte (README + código + evidencia).

7. Flujo de n8n #1 — Generación de la rutina inicial

Archivo: `src/flujo/1-rutina-inicial.json`

Qué hace en conjunto: cuando un usuario rellena el formulario inicial, este flujo genera su rutina con IA, crea una hoja de seguimiento personal para él, se la comparte y le envía todo por correo. Además registra al cliente en la 'libreta' para poder encontrarlo después.

Idea clave: se le pide a la IA que devuelva la rutina como **JSON estructurado** (datos ordenados), no como texto. Así el mismo resultado sirve para dos cosas: armar el correo y llenar la hoja. Antes de usarlo, un nodo de código valida ese JSON (esto es un 'guardrail': si la IA falla, el flujo se detiene con un error claro).

Nodo por nodo:

Nodo	Qué hace	Por qué / para qué
Nueva fila en Sheet	Se dispara cuando llega una fila nueva a la pestaña 'general'.	Es el gatillo del flujo: conecta el formulario con la automatización, sin intervención manual.
Generar rutina (JSON) + Gemini	Envía los datos del usuario a Gemini y le pide la rutina en formato JSON.	La IA crea la rutina personalizada. El JSON permite reutilizar los datos en pasos siguientes.
Preparar datos (Code)	Valida el JSON (que tenga días válidos) y arma las filas de ejercicios y los textos.	Guardrail: evita usar una respuesta inválida. Prepara los datos para la hoja y el correo.
Crear planilla (Sheets)	Crea una hoja de cálculo nueva para ese cliente.	Cada cliente necesita su propia hoja privada para registrar sus pesos.
Expandir filas (Code)	Convierte el arreglo de ejercicios en un ítem por ejercicio.	El nodo de escritura necesita un ítem por cada fila que va a escribir.
Escribir ejercicios (Sheets)	Escribe todos los ejercicios en la hoja del cliente.	Deja la hoja lista, con columnas vacías para que el cliente anote peso, reps y dificultad.
Armar correo (Code)	Construye el HTML del correo con la rutina y el enlace a la hoja.	Controla la presentación por código para que el correo se vea siempre igual y bien.
Compartir hoja (Drive)	Hace la hoja editable por enlace.	El cliente es otra persona: necesita acceso para poder abrir y editar su hoja.
Enviar correo (Gmail)	Envía la rutina al correo del usuario.	Es el touchpoint de entrega: el usuario recibe su plan y el enlace a su hoja.
Registrar cliente (Sheets)	Guarda en 'hojas_clientes' el correo y el ID de la hoja creada.	Es la 'libreta' que permite al flujo de seguimiento encontrar la hoja del cliente después.

8. Flujo de n8n #2 — Seguimiento y ajuste

Archivo: `src/flujo/2-seguimiento.json`

Qué hace en conjunto: cuando el cliente ya entrenó y registró sus pesos, y rellena el formulario de seguimiento, este flujo busca su hoja, lee los datos reales que anotó, y le pide a la IA que ajuste las cargas (sube donde le fue fácil, baja donde le costó). Luego le envía la rutina ajustada por correo.

Idea clave (el loop): aquí se cierra el círculo. El sistema no adivina: usa los **datos reales** del cliente para mejorar su rutina. Esto es lo que diferencia a FORJA de una app de rutinas cualquiera.

Nodo por nodo:

Nodo	Qué hace	Por qué / para qué
Nuevo seguimiento en Sheet	Se dispara con una fila nueva en la pestaña 'seguimiento'.	Gatillo del flujo: reacciona cuando el cliente pide un ajuste.
Leer libreta (Sheets)	Lee todas las filas de 'hojas_clientes'.	Trae el mapa correo → hoja para poder ubicar la planilla del cliente.
Buscar planilla del cliente (Code)	Filtra la libreta por el correo del cliente y obtiene el ID de su hoja.	Sin esto, n8n no sabría cuál de todas las hojas es la del cliente.
Leer registros del cliente (Sheets)	Abre la hoja del cliente y lee los pesos, reps y dificultad que anotó.	Son los datos reales que alimentan el ajuste de la IA.
Preparar contexto (Code)	Arma un resumen de texto con esos registros y el feedback del formulario.	Le da a la IA la información ordenada para que ajuste con criterio.
Ajustar rutina (Gemini)	La IA ajusta las cargas y ejercicios según los datos reales.	Es el corazón del seguimiento: sube/baja pesos según cómo le fue al usuario.
Dar formato al correo (Code)	Envuelve el contenido que devuelve la IA en una plantilla fija.	Separa contenido (IA) de presentación (código): el correo se ve siempre consistente.
Enviar rutina ajustada (Gmail)	Envía la rutina ajustada al correo del cliente.	Cierra el ciclo: el usuario recibe su plan mejorado.

9. Flujo de n8n #3 — Alertas de error (Error Trigger)

Archivo: `src/flujo/3-error-trigger.json`

Qué hace en conjunto: es un flujo 'vigilante'. No lo dispara un formulario, sino el **fallo de otro flujo**. Si el flujo de rutina inicial o el de seguimiento fallan, este envía un correo de alerta al administrador con los detalles del error.

Nodo por nodo:

Nodo	Qué hace	Por qué / para qué
Error Trigger	Se dispara automáticamente cuando otro workflow (configurado para avisarle) falla.	Permite enterarse de los fallos sin estar mirando n8n todo el día.
Avisar por correo (Gmail)	Envía un correo con el nombre del flujo que falló, el error y el nodo afectado.	El administrador recibe la alerta y puede reaccionar rápido.

10. Manejo de errores — los 3 mecanismos

La solución implementa los tres mecanismos de robustez requeridos:

- **Retries (reintentos):** el nodo de la IA reintentará automáticamente si la llamada a la API falla por un problema temporal (por ejemplo, saturación del servicio).
- **Continue on Fail (continuar ante error):** los nodos clave están configurados en 'On Error: Continue', de modo que si un caso concreto revienta, el flujo sigue procesando los demás en vez de detenerse por completo.
- **Error Trigger (disparador de error):** el flujo #3, que avisa por correo cuando algo falla.

11. Decisiones técnicas clave

Preguntas típicas y sus respuestas, por si se defienden en la presentación:

- **¿Por qué la IA devuelve JSON?** Para separar el contenido (lo que dice la IA) de la presentación (el correo y la hoja los arma el código). Así el resultado es consistente y reutilizable, y se puede validar (guardrail).
- **¿Por qué se usan nodos nativos de Google y no llamadas HTTP directas?** Porque n8n bloquea el uso de las credenciales de Google gestionadas por n8n en el nodo HTTP genérico; los nodos nativos (Sheets/Drive) sí las aceptan.
- **¿Por qué una hoja por cliente?** Para que cada persona registre su progreso de forma privada y la IA pueda leerlo de vuelta para ajustar la rutina.
- **¿Por qué Google Gemini?** Tiene una capa gratuita, ideal para un MVP universitario sin costos.
- **¿Por qué el correo lo arma un nodo de código y no la IA directamente?** Para garantizar un diseño fijo y consistente en cada envío, sin depender de que la IA lo formatee bien cada vez.

12. Entregable y evidencia

El repositorio de la entrega contiene:

- **README.md:** documento maestro con las 10 secciones (identificación, problema/solución, arquitectura, las 4 verticales, touchpoint, cómo correrlo, análisis de negocio del Track B, limitaciones y roles).

- **src/**: los 3 flujos de n8n, los prompts de la IA (uno por archivo), el Apps Script y la estructura de la BBDD.
- **evidencia/**: capturas de pantalla, el video de demo (enlace a YouTube) y las conversaciones de la prueba de demanda.
- **docs/**: esta documentación y la guía de conversaciones.

13. Glosario rápido

- **n8n**: herramienta de automatización visual basada en 'nodos' que conectan servicios.
- **Nodo**: cada paso o bloque dentro de un flujo de n8n.
- **Trigger (disparador)**: el nodo que inicia un flujo (aquí, una fila nueva en el Sheet).
- **LLM / Gemini**: el modelo de Inteligencia Artificial que genera y ajusta las rutinas.
- **JSON**: un formato de datos ordenado que permite que el programa reutilice la respuesta de la IA.
- **Guardrail**: una validación que limita o revisa la salida de la IA antes de usarla.
- **Touchpoint**: el punto por donde el usuario interactúa con la solución (aquí, la landing y el correo).

14. Preguntas posibles del Q&A; (con respuestas)

Set de preguntas que el profesor podría hacer sobre la arquitectura y las decisiones, con una respuesta breve para defender cada una. La regla del curso: usar IA para construir está bien, pero hay que entender lo que se entrega.

P: ¿Por qué eligieron n8n en vez de programar un backend desde cero?

R: Porque n8n permite orquestar servicios (Google Sheets, Gmail, la IA) de forma visual, con manejo de errores integrado, sin montar y mantener un servidor propio. Para un MVP es más rápido, más claro de explicar y suficientemente robusto.

P: ¿Por qué usan Google Sheets como base de datos y no Postgres o Supabase?

R: Porque para el volumen de un MVP, Sheets cumple: almacena los datos del negocio, es gratis, y además sirve directamente como disparador de n8n. Reconocemos que a mayor escala convendría una base de datos real (Postgres o Supabase con seguridad por fila); lo dejamos como próximo paso.

P: ¿Por qué le piden a la IA que devuelva JSON y no directamente el texto del correo?

R: Para separar el contenido (lo que decide la IA) de la presentación (el correo y la hoja los arma nuestro código). Con datos estructurados podemos reutilizar la misma respuesta para dos cosas y, sobre todo, validarla antes de usarla.

P: ¿Qué es el 'guardrail' y dónde está en su flujo?

R: Es una validación que revisa la salida de la IA antes de usarla. En nuestro flujo es un nodo de código que hace `JSON.parse` y verifica que la rutina tenga días válidos; si no, se detiene con un error claro en vez de continuar con datos corruptos.

P: ¿Por qué usan los nodos nativos de Google y no el nodo HTTP genérico?

R: Porque n8n, por seguridad, bloquea el uso de las credenciales de Google gestionadas por n8n dentro del nodo HTTP genérico. Los nodos nativos de Google Sheets y Google Drive sí aceptan esas credenciales, así que fueron la vía correcta para crear la planilla, escribir en ella y compartirla.

P: ¿Por qué crean una hoja de cálculo nueva por cada cliente?

R: Para que cada persona registre su progreso de forma privada y separada, y para que la IA pueda leer exactamente los datos de ese cliente al hacer el ajuste. Una sola hoja compartida mezclaría datos de todos.

P: ¿Cómo encuentra el flujo de seguimiento la hoja correcta del cliente?

R: Cuando se crea la rutina inicial, guardamos en una pestaña 'libreta' (hojas_clientes) el mapa correo -> ID de su hoja. En el seguimiento, buscamos por el correo del cliente en esa libreta y obtenemos el ID de su planilla.

P: ¿Por qué eligieron Google Gemini y no otro modelo?

R: Principalmente por su capa gratuita, ideal para un proyecto universitario sin costos. La arquitectura es agnóstica: si hiciera falta, se podría cambiar por otro modelo con poco esfuerzo.

P: ¿Cómo manejan los errores?

R: Con los tres mecanismos: Retries (la IA reintenta si la API falla), Continue on Fail (si un caso revienta, el flujo sigue con los demás) y un Error Trigger (un flujo aparte que nos avisa por correo

cuando algo falla).

P: ¿Qué pasa si la IA inventa datos o devuelve algo inválido?

R: El nodo de validación detiene el flujo si el JSON no es válido. Además, en el prompt le damos reglas explícitas (no dar kilos exactos, respetar lesiones, un número de días exacto), lo que limita su comportamiento.

P: ¿Cómo aseguran que la rutina respete las lesiones del usuario?

R: El formulario captura las lesiones y se las pasamos a la IA en el prompt con la instrucción explícita de respetarlas y evitar ejercicios peligrosos. En el seguimiento también se consideran nuevas molestias.

P: ¿Por qué el correo lo arma un nodo de código y no la IA?

R: Para garantizar un diseño fijo y consistente en cada envío. La IA aporta el contenido (los ejercicios y ajustes) y el código aplica la plantilla, así el correo siempre se ve igual de bien.

P: ¿El seguimiento se puede repetir? ¿Qué limitación tiene?

R: Sí: el cliente actualiza sus pesos y vuelve a enviar el formulario cuantas veces quiera. La limitación actual es que la rutina ajustada llega por correo pero no se reescribe en la hoja; lo tenemos identificado como próximo paso.

P: ¿Cuál es su métrica north-star y por qué?

R: El número de seguimientos completados por usuario al mes. La elegimos porque mide a la vez retención (que vuelvan) y valor entregado (usan justo lo que los hace pagar el plan Pro).

P: ¿Cómo escalaría esto con miles de usuarios?

R: Habría que migrar de Google Sheets a una base de datos real, revisar los límites de la API de la IA y de Gmail, y automatizar el cobro de la suscripción. La arquitectura por flujos facilita ir reemplazando piezas sin rehacer todo.